Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: D.N Shirshika **Reg. No:192425396**

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis

- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for k = 2, 4, 8, with and without a local sensor), with code and output as proof.

MARK DISTRIBUTION – ASSIGNMENT_1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

1. Introduction

Artificial Intelligence (AI) is a branch of computer science that enables machines to perform tasks that normally require human intelligence. These tasks include problem-solving, decision-making, learning and searching.

In this project, an intelligent medicine-delivery robot is used in a multi-block hospital. The robot carries medicines from the central pharmacy to different wards through corridors, junctions and lifts.

The hospital is represented as a graph in which:

- Nodes represent hospital locations.
- Edges represent connections between locations.
- Edge weights represent the time required to travel between locations.
- The pharmacy is the starting point.
- The requested ward is the goal.

The robot uses search algorithms to identify a suitable path while avoiding unnecessary movements and reducing delivery time.

Main objective: To design and evaluate efficient search strategies that can work when the hospital grows from 1,000 to more than 1,000,000 states.

TASK 1 – Design the Agent and Search Algorithms

An intelligent agent is a system that observes its environment through sensors and performs actions using actuators to achieve a specific goal.

The hospital robot follows a rational-agent approach because it selects actions based on the available information and tries to achieve the best possible result.

The agent works through four major steps:

1. Perception – collects information using sensors.
2. Decision-making – analyses the current situation.
3. Action selection – chooses the next movement.
4. Execution – moves towards the destination.

Environment explanation:

The hospital environment is dynamic because patients, nurses and other robots may move through corridors. It is partially observable if the robot cannot detect every obstacle or its exact location.

TASK 2 – Identify the Basic Operation

The basic operation is the most frequently executed operation that influences the running time of an algorithm.

comparison:

Factor	BFS	DFS	IDS
Main operation	Queue removal and expansion	Recursive calls	Repeated node expansion
Data structure	Queue	Stack / recursion	Depth-limited recursion
Exploration	Level-wise	Depth-wise	Increasing depth
Repetition	Usually avoids visited nodes	Usually avoids visited nodes	Re-explores nodes

Importance:

Identifying the basic operation helps determine how much computational work is performed as the hospital map increases in size.

For example, a hospital with 1,000 locations may require fewer node expansions than a hospital with 100,000 locations.

TASK 3 – Mathematical Complexity Analysis

Additional mathematical explanation:

The branching factor determines how many new locations can be reached from one location.

Suppose each hospital junction has three connected corridors.

Then:

\[b=3 \]

The number of nodes at each level is:

Level	Number of nodes
0	1
1	3
2	9
3	27
4	81
5	243

Total nodes:

\[N=1+3+9+27+81+243 \]

\[N=364 \]

This demonstrates how the number of possible search nodes increases with depth.

Additional complexity comparison:

Algorithm	Time	Search Space
BFS	$O(b^d)$	$O(b^d)$
DFS	$O(b^m)$	O(bm)
IDS	$O(b^d)$	O(bd)

Preprocessing must also be included. For an adjacency-list graph:

\[O(V+E) \]

This represents the cost of constructing and storing the hospital map.

TASK 4 – Best, Average and Worst Cases

Explanation with an example:

Suppose the pharmacy is connected to Junction A and Junction B.

- Best case: The requested ward is found immediately.
- Average case: The robot checks several junctions before reaching the ward.
- Worst case: The robot explores almost every reachable location before finding the destination or confirming that it is unreachable.

Practical importance:

Best-case analysis helps understand ideal performance, average-case analysis explains normal operation, and worst-case analysis helps identify possible delays.

In a hospital, worst-case performance is important because medicine delivery may be time-sensitive.

TASK 5 – Asymptotic Analysis

Explanation:

Asymptotic analysis helps predict how an algorithm performs when the number of hospital locations becomes very large.

For example, consider a search tree with branching factor 2.

Depth	Total nodes
1	3
2	7
3	15
4	31
5	63
6	127

The number of nodes increases exponentially with depth.

Interpretation:

- BFS can become memory-intensive.
- DFS can explore unnecessary deep paths.
- IDS provides a balance between memory requirements and shallow-solution discovery.

Big O describes an upper bound, Omega describes a lower bound and Theta describes a tight asymptotic bound.

TASK 6 – Recurrence Relation

Explanation:

A recurrence relation describes how the execution time of a recursive algorithm depends on smaller problems.

In DFS, one node may generate multiple recursive calls.

For branching factor b:

$$T(n)=bT(n-1)+c$$

Here, c represents the work performed at the current node, including checking the goal and generating successors.

Example:

If b = 2:

$$T(n)=2T(n-1)+c$$

Each recursive level can generate twice as many calls as the previous level.

This explains why recursive search can grow exponentially.

TASK 7 – Solve the Recurrence Relation

Explanation using a recursion tree:

For:

$$T(n)=2T(n-1)+c$$

At each level, the number of recursive calls doubles.

Level	Number of calls
0	1
1	2
2	4
3	8
4	16

Total calls:

$$1+2+4+8+16$$

Therefore, the number of calls grows exponentially.

$$T(n)=O(2^n)$$

For general branching factor b:

\[T(n)=O(b^n) \]

TASK 8 – Implement and Test Algorithms

Implementation explanation:

The Python program represents each hospital location as a node and each corridor as a connection.

The three algorithms use different data structures:

- BFS uses a queue.
- DFS uses recursion.
- IDS uses depth-limited recursion.

Test cases:

Test Case	Description	Expected Result
1	Pharmacy to Ward 5	Route found
2	Pharmacy to disconnected ward	No route
3	Pharmacy to itself	Empty movement route
4	Large graph	Search performance recorded
5	Multiple possible paths	Algorithm returns a valid path

Validation:

The returned route should be checked to confirm that every consecutive pair of locations is connected in the hospital graph.

The experiment should use the same graph, starting point, destination and random seed for all three algorithms.

TASK 9 – Experimental Performance

parameters to measure:

1. Execution time – how long the algorithm takes.
2. Nodes generated – how many nodes are created.
3. Nodes expanded – how many nodes are explored.
4. Maximum frontier – the maximum number of pending nodes.
5. Path length – number of movements required.
6. Path cost – total travel time.

Importance of experimental analysis:

Theoretical analysis predicts algorithm performance, but experimental analysis demonstrates how the algorithm behaves when executed in Python.

Run each experiment five times and calculate the average execution time.

TASK 10 – Compare Theory with Experimental Behaviour

Discussion:

Theoretical and experimental results may not always be exactly the same.

For example, BFS may have higher memory usage because it stores multiple pending nodes. DFS may use less memory but take longer if it explores an incorrect corridor first.

IDS may perform additional node expansions because it repeats searches with increasing depth limits.

Factors affecting results:

- Processor speed.
- Available RAM.
- Python execution overhead.
- Graph connectivity.
- Number of duplicate states.
- Goal location and search order.

Therefore, both mathematical analysis and experimental results are important when selecting an algorithm.

TASK 11 – Recommend and Defend the Final Design

Justification:

IDS is selected for the initial design because it combines the low memory usage of DFS with the ability to find shallow solutions like BFS.

Benefits for hospital operations:

- Reduces search-memory requirements.
- Finds shallow routes.
- Avoids storing a very large BFS frontier.
- Can be applied to different hospital layouts.
- Provides a simple approach for route exploration.

Future improvement:

The robot can be improved by introducing Uniform-Cost Search to consider different corridor travel times, and sensor-based obstacle detection to handle changing hospital conditions.

This would make the medicine-delivery system more suitable for real-world hospital environments.

TASK 12 – Re-evaluate and Adapt the Design

Explanation:

A localisation sensor helps a robot identify its current position. If this sensor fails, the robot cannot confidently decide which corridor or junction it is currently occupying.

This creates a partially observable environment.

Example:

Suppose the robot may be at three locations:

$$ B=\{A,B,C\} $$

After receiving an observation from a local sensor, it may identify that it is at location B.

The updated belief state becomes:

$$ B'=\{B\} $$

This reduces uncertainty.

Advantages of the modified system:

1. Handles sensor failures.
2. Maintains multiple possible locations.
3. Updates its belief using observations.
4. Reduces unnecessary movements.
5. Improves reliability during medicine delivery.

Future enhancement:

The hospital robot can use cameras, LiDAR sensors and landmark detection to identify corridors and junctions. Combining these technologies with belief-state search can improve localisation and navigation.

CONCLUSION

The Hospital Medicine-Delivery Robot demonstrates how Artificial Intelligence and search algorithms can be used to solve real-world navigation problems. In this assignment, three uninformed search algorithms, namely Breadth-First Search (BFS), Depth-First Search (DFS) and Iterative Deepening Search (IDS), were designed and analysed for finding routes from the central pharmacy to different hospital wards.

The time complexity, space complexity, recurrence relations, best-case, average-case and worst-case behaviours of these algorithms were studied. Python implementation and experimental testing help evaluate their performance as the hospital grows from 1,000 to 1,000,000 or more states.

Based on the analysis, IDS provides a suitable balance between memory usage and search efficiency when finding shallow solutions. The Day-4 design challenge further demonstrates the importance of handling uncertain locations through belief-state search and local sensing when the localisation sensor fails.

Overall, this assignment provides a practical understanding of intelligent agents, problem formulation, uninformed search strategies and algorithm analysis. The proposed system can help reduce unnecessary movement, improve medicine-delivery efficiency and support reliable hospital operations.